



Local Model Creation
– Cameron Schaff

Here's the write-up for the Local Model Creation lab, styled to match the PA-220 lab document:

Schaff, Cameron @CCNP&CYBERSECURITY CCNP IN PROGRESS -_- Cameron Schaff

Local Model Creation & Training on Consumer Hardware

Cameron Schaff

Cybersecurity Lab Report: Local LLM Pretraining and Fine-Tuning on a ThinkStation P7

Purpose

Train a large language model from scratch on local consumer-grade hardware using distributed GPU computing, enabling offline AI inference without reliance on cloud providers.

Background Information on Lab Concepts

Concept: Understand distributed training with PyTorch DDP (Distributed Data Parallel)

Concept: Multi-GPU coordination using NCCL over PCIe

Concept: Model pretraining, tokenization, and supervised fine-tuning pipelines

Background:

Features of the Training Stack

- **Distributed Data Parallel (DDP) via torchrun** Through PyTorch's torchrun launcher and NCCL backend, training workload is split evenly across both GPUs. Each GPU maintains a full model copy and gradients are synchronized after each backward pass, effectively doubling throughput compared to single GPU training.
- **NCCL — NVIDIA Collective Communications Library** NCCL handles the low-level GPU-to-GPU communication over PCIe. On Linux this is the preferred backend over

Gloo for CUDA devices, providing significantly lower latency and higher bandwidth during the all-reduce gradient synchronization step.

- **Gradient Checkpointing** Rather than storing all intermediate activations in VRAM during the forward pass, gradient checkpointing recomputes them during the backward pass. This trades compute time for memory, allowing a ~984M parameter model to fit within 24GB of VRAM per GPU.
- **Mixed Precision Training (FP16)** Weights are stored and computed in 16-bit floating point where possible, with a GradScaler managing numeric stability. This halves VRAM consumption for activations and increases throughput on Tensor Cores.
- **BPE Tokenization via SentencePiece** Text is encoded into subword token IDs using a Byte Pair Encoding model trained on the corpus. A 32,000 token vocabulary balances coverage and embedding table size for a ~1B parameter model.
- **Supervised Fine-Tuning (SFT)** After pretraining on raw text, the model is further trained on structured prompt-response pairs to teach instruction-following behavior. A loss mask ensures only the assistant response tokens contribute to the gradient, preserving the pretraining knowledge while shaping the model's output style.

Compared to cloud-based training that costs thousands of dollars per run and provides no hardware ownership. Local training on owned hardware allows full control over data, checkpointing, and model weights with zero ongoing cost after initial hardware investment.

Lab Summary

- Prepare raw corpus data and train a BPE tokenizer
- Encode the corpus into binary token IDs for fast loading
- Pretrain a transformer model using 2-GPU DDP via torchrun
- Fine-tune the pretrained model on SFT examples
- Run interactive inference via CLI chat interface

Full Guide

Hardware Used

- Lenovo ThinkStation P7

- 2x NVIDIA Titan RTX (24GB VRAM each)
- Intel Xeon w5-3435X (32 cores)
- 128GB DDR5 ECC RAM
- Pop!_OS 26.04 (NVIDIA ISO)

Step 1: Environment Setup

The project runs entirely inside a Docker container to avoid dependency conflicts with the host system. Prerequisites on the host are minimal.

Install Docker and add your user to the docker group:

```
sudo apt install docker.io
sudo usermod -aG docker $USER
newgrp docker
```

Install the NVIDIA Container Toolkit to allow GPU passthrough into containers. This requires adding NVIDIA's repository manually as it is not included in the default Pop!_OS apt sources:

```
sudo apt install curl
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

```
sudo apt update && sudo apt install nvidia-container-toolkit
sudo systemctl restart docker
```

Verify GPU passthrough is functional before proceeding:

```
docker run --rm --gpus all nvidia/cuda:12.1.0-base-ubuntu22.04 nvidia-smi
```

A successful output lists both Titan RTX cards with their VRAM and CUDA version. If you receive a driver/library version or mismatch error, a system reboot resolves that kernel module mismatch.

Step 2: Build the Training Container

The project includes a build script that pulls a PyTorch 2.3.0 CUDA 12.1 base image and installs all Python dependencies inside the container. This is approximately 5GB on first run and cached for subsequent builds.

```
cd ~/Downloads/tinygpt-lab-popos/tinygpt-lab
chmod +x build.sh run.sh
./build.sh
```

The build script verifies Docker and NVIDIA-SMI prerequisites, builds the image tagged as tinygpt-lab:latest, and runs a GPU detection check. Both GPUs should be reported with their full 24576 MiB VRAM allocation.

Step 3: Data Preparation

Raw text files are placed in data/raw/. The pipeline accepts .txt and .md files and supports large sources including Wikipedia dumps and OpenWebText. The data preparation script cleans, merges, and writes a single corpus.txt.

The tokenizer is trained on the corpus using SentencePiece BPE with a 32,000 token vocabulary. This is a one-time operation and the resulting model file is reused for all subsequent encoding and inference.

Corpus encoding converts the full text corpus into a binary file of uint16 token IDs (pretrain_ids.bin). This step uses a parallel multiprocessing encoder that splits the corpus into chunks by byte offset, snapping each boundary to the nearest document separator. Workers stream and flush every 2 million tokens to keep RAM usage flat regardless of corpus size. A live progress bar reports throughput and estimated time remaining.

On a 40GB corpus with 30 parallel workers, encoding completes in approximately 15–20 minutes at 300 MB/s sustained disk read speed.

Step 4: Pretraining

The model architecture is a decoder-only transformer with the following configuration for the full ~984M parameter run:

Parameter	Value
Layers	24
Attention Heads	16
Model Dimension	1792
Feed-Forward Dimension	7168
Vocabulary Size	32,000
Context Length	1024 tokens

Training is launched through the run script which calls torchrun internally:

```
docker run -u 1000:1000 --gpus=all \  
-e HOME=/project \  
-e PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128 \  
-it --rm \  
-v "/home/labuser/Downloads/tinygpt-lab-popos/tinygpt-lab:/project" \  
-w /project tinygpt-lab:latest python run.py
```

The PYTORCH_CUDA_ALLOC_CONF override disables expandable_segments which causes an internal CUDA allocator assertion failure on PyTorch 2.3.0. Setting max_split_size_mb:128 is the correct replacement for this hardware configuration.

Torchrun initializes two processes, one per GPU, using NCCL as the communication backend. Each process loads its own shard of the dataset using a DistributedSampler. Gradients are accumulated over 16 micro-steps before a synchronized all-reduce across both GPUs, giving an effective batch size of 65,536 tokens per step.

The learning rate follows a cosine decay schedule with a warmup period. Training progress is logged to the terminal and written to a JSON status file readable by the dashboard:

Step	Loss	Val	LR	Tok/s	ETA
25	9.1035	—	9.60e-06	9.9k	184h00m
1,000	4.9070	4.9465	2.00e-04	9.9k	173h43m
5,000	3.5823	3.6156	1.99e-04	9.9k	166h11m
25,000	3.0836	3.1347	1.74e-04	9.9k	131h08m
50,000	2.9444	2.9581	1.11e-04	9.8k	87h23m

Validation loss is evaluated every 1,000 steps over 100 randomly sampled validation batches to avoid the full 450M token validation set causing training to hang. Best checkpoints are saved automatically when validation loss improves.

The target loss for fine-tuning readiness is approximately 2.5. At 10,500 tokens per second across both GPUs, the full 100,000 step run completes in approximately 7 days.

A real-time training dashboard can be launched in a second terminal during training:

```
cd ~/Downloads/tinygpt-lab-popos/tinygpt-lab && python3 gui/dashboard.py
```

Step 5: Supervised Fine-Tuning

Once pretraining reaches an acceptable loss, the best checkpoint is used as the starting point for supervised fine-tuning. SFT examples are structured prompt-response pairs in JSONL format stored at `data/sft_data.jsonl`.

The fine-tuning trainer applies a loss mask so that only the assistant response tokens contribute to the gradient. This allows the model to retain its pretraining knowledge while learning the instruction-following format. A lower learning rate (5e-6) and gradient clipping (0.5) reduce catastrophic forgetting of pretrained knowledge.

Fine-tuning converges rapidly on small datasets. With 1,100 SFT examples the loss drops below 1.0 within a few hundred steps, after which early stopping prevents overfitting.

Step 6: Inference

Interactive chat with the trained model runs inside the same Docker container:

```
docker run -u 1000:1000 --gpus=all \  
-e HOME=/project \  
-e PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128 \  
-it --rm \  
-v "/home/labuser/Downloads/tinygpt-lab-popos/tinygpt-lab:/project" \  
-w /project tinygpt-lab:latest python inference/chat.py
```

The inference engine loads the fine-tuned checkpoint and runs autoregressive generation with temperature sampling, top-k filtering, top-p nucleus sampling, and a repetition penalty window to suppress looping output. Type /help inside the chat for available commands including adjusting generation parameters at runtime.

Troubleshooting Notes

If training crashes at the backward pass with `!block->expandable_segment_ INTERNAL ASSERT FAILED`, the `expandable_segments:True` environment variable baked into the Dockerfile is incompatible with PyTorch 2.3.0. Override it at container launch with `-e PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128`.

If resuming from a checkpoint fails with shape mismatches, the checkpoint was saved from a different model scale than the current config. Delete the checkpoint directory and retrain from scratch, or ensure the config matches the checkpoint's architecture exactly.

If the dataset encoding step runs out of memory with many workers, reduce the worker count in `training/dataset.py`. Eight workers with 16MB read buffers and 2M token flush intervals keeps peak RAM under 30GB on a 128GB system.

COMMIT CHANGES!!!

Unlike normal CCNP Labs saving is not as easy as pressing Commit. — but the equivalent is equally important. Always verify that your latest `pretrain_best.pt` checkpoint has been

saved before stopping training. The trainer saves checkpoints every 5,000 steps and overwrites `pretrain_best.pt` whenever validation loss improves.

Good luck with your continued model training!

Schaff, Cameron @NewportCiscoCCNP

Credits: Claude Code · Codex · HuggingFace · Wikipedia · JetBrains IDE · Concept inspired by a Cursor IDE + Opus 4.7 demonstration on YouTube